

Customer Onboarding AI

Task 4 — Controlled UISpec Renderer

Beginner-Friendly Enterprise Engineering Guide

Task 4 follows the Task 3 deterministic validator and the architecture review. Its purpose is to create the controlled UI Runtime boundary that renders only an already-approved UISpec.

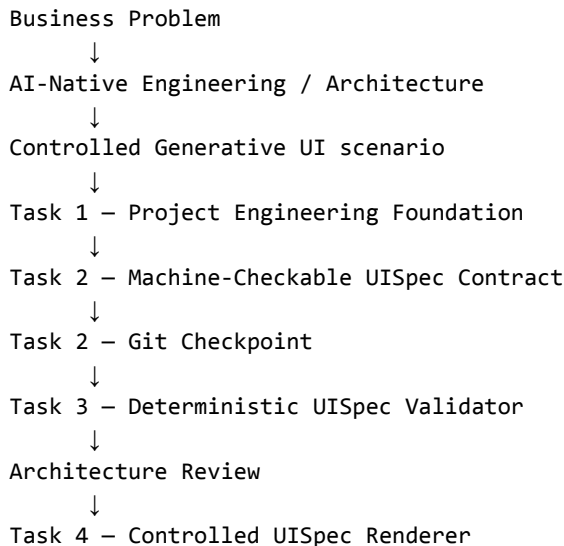
Executive Summary

Task 4 implements the controlled React + TypeScript renderer for the v0 UISpec contract. The central security and architecture principle is that the renderer does not accept raw AI output. It accepts only an already-approved UISpec and resolves component names through an explicit, developer-controlled registry.

The implementation was reviewed against the Task 4 requirements. The implementation is suitable for the next Git checkpoint, with one important clarification: the Task 3 Python validator and the Task 4 React renderer are not yet physically connected through a runtime API. That is intentional at this stage. The architecture defines the future flow, but Task 4 implements only the renderer boundary.

1. Where Task 4 Fits in the Learning Sequence

The project is intentionally being built in small increments rather than creating the whole application at once.



This sequence demonstrates specification-driven and AI-native engineering: define the contract first, create deterministic controls, then build the runtime capability that is allowed to consume the approved result.

2. Task 4 Objective

Build the controlled UISpec renderer for the UI Runtime.

The renderer must accept only an already-approved UISpec.

Correct:

```
render(approvedUISpec)
```

Not:

```
render(rawAIOutput)
```

This distinction is fundamental. The renderer is not responsible for deciding whether AI-generated data complies with the model contract. That responsibility belongs to the deterministic validation layer.

3. The Task 4 Prompt

The following is the Task 4 implementation brief preserved in the project walkthrough. It is explicitly a reconstructed implementation brief based on the preserved Task 4 requirements, not a verbatim transcript of the original prompt.

Proceed with Task 4: implement the Controlled UISpec Renderer.

This is the next smallest implementation increment for the Customer Onboarding AI application.

IMPORTANT ARCHITECTURAL DECISION:

The renderer must receive ONLY an already-approved/validated UISpec.

The renderer must NOT accept raw AI/model output.

Conceptually:

```
RAW AI OUTPUT
  ↓
UISpec Validator
  ↓
APPROVED UISpec
  ↓
Controlled Renderer
  ↓
Approved React Components
  ↓
Browser UI
```

The renderer contract should therefore be conceptually:

```
render(approvedUISpec)
```

Do NOT implement:

```
render(rawAIOutput)
```

Before making changes:

1. Inspect the current repository.
2. Read:
 - CLAUDE.md

- README.md
- docs/business-problem.md
- docs/architecture.md
- specs/spec.md
- specs/model-contracts.md
- specs/schemas/ui-spec.schema.json
- domain/validator.py
- tests/domain/test_validator.py

3. Review the existing Task 1, Task 2 and Task 3 implementation before changing anything.

TASK 4 GOAL

Implement the deterministic Controlled UISpec Renderer.

The renderer must translate an approved UISpec into known application UI components.

Use an explicit component registry.

The v0 approved component vocabulary is:

- Hero
- Heading
- Text
- FeatureCard
- FeatureGrid
- CTA
- Alert
- InfoPanel
- ProgressIndicator

The renderer must map these component types to known React components.

IMPORTANT SECURITY / CONTROL REQUIREMENTS

The model must NOT be able to create arbitrary React components.

The renderer must NOT:

- execute generated JavaScript
- execute shell commands
- use eval()
- use Function()
- interpret model output as executable code
- inject arbitrary executable HTML
- dynamically import a component based on an arbitrary model-provided string
- bypass the component registry
- silently render unsupported component types

The model is only allowed to describe UI using the approved UISpec contract.

RENDERER DESIGN

Create an explicit registry along the lines of:

```
componentRegistry = {
  Hero: HeroComponent,
  Heading: HeadingComponent,
  Text: TextComponent,
  ...
}
```

```
}
```

Use an explicit switch/registry approach rather than dynamic component resolution.

If a component type is not present in the approved registry, rendering must fail deterministically.

Do not silently substitute another component.

Do not auto-repair invalid UISpec.

VALIDATION BOUNDARY

Do not duplicate the entire JSON Schema inside the renderer.

The renderer should assume that its input has already passed the deterministic validator.

However, it should still defend its own registry boundary.

The architectural responsibility should remain:

JSON Schema



Structural validation

Validator



Runtime/cross-field validation

Renderer



Approved component mapping

REACT IMPLEMENTATION

If the repository does not yet contain a React application, create ONLY the minimum React/TypeScript structure needed to demonstrate and test the renderer.

Do not build the complete Customer Onboarding application yet.

Do not build the full landing page as a product feature yet.

Keep the implementation intentionally small.

The renderer should be able to render a representative approved UISpec such as:

```
{
  "version": "0.1",
  "meta": {
    "generated_for": {
      "segment": "...",
      "product_interest": "...",
      "region": "..."
    }
  },
  "root": {
    "type": "Hero",
    "props": {
      "...": "..."
    },
    "children": [
```

```

    {
      "type": "Heading",
      "props": {
        "...": "..."
      }
    },
    {
      "type": "Text",
      "props": {
        "...": "..."
      }
    },
    {
      "type": "CTA",
      "props": {
        "...": "..."
      }
    }
  ]
}

```

Use the ACTUAL existing model contract and schema as the source of truth for exact properties. Do not invent properties.

TESTING

Add deterministic tests for the renderer.

At minimum test:

1. Each approved component type resolves through the registry.
2. A valid Hero with valid children renders successfully.
3. A valid FeatureGrid with FeatureCard children renders successfully.
4. A valid CTA renders successfully.
5. An unsupported component type is rejected.
6. The renderer does not dynamically execute model-provided code.
7. Unknown component properties do not become arbitrary DOM attributes or executable behavior.
8. The same approved UISpec produces the same renderer result.
9. The renderer does not mutate the input UISpec.
10. A representative approved UISpec can be rendered through the complete registry path.

Keep tests readable and beginner-friendly.

IMPORTANT:

Do not add AI/model integration.

Do not add FastAPI integration.

Do not add authentication.

Do not add document processing.

Do not add customer registration.

Do not add risk scoring.

Do not add human approval workflow.

Do not add production infrastructure.

Do not add unnecessary dependencies.

Do not redesign the existing model contract.

Do not modify the JSON Schema unless you discover a genuine inconsistency required by the renderer. If such an inconsistency is discovered, STOP and report it rather than silently changing the contract.

BEFORE IMPLEMENTATION

First explain:

1. What renderer structure you intend to create.
2. Where the component registry will live.
3. How the approved UISpec will flow through the renderer.
4. What files you intend to create or modify.
5. How the renderer remains deterministic.
6. How the renderer prevents arbitrary model-generated code from executing.

Then implement the task.

AFTER IMPLEMENTATION

Run the renderer tests.

Run the existing Task 3 validator tests.

Run the existing Task 2 schema verification.

Confirm:

- all existing tests still pass
- renderer tests pass
- no model integration was added
- no FastAPI integration was added
- no authentication was added
- no document/risk/approval functionality was added
- no arbitrary component execution is possible
- the renderer accepts approved UISpec rather than raw model output

Review the git diff.

Do NOT commit.

Do NOT push.

Do NOT start Task 5.

Stop and report:

1. Files created
2. Files modified
3. Component registry design
4. Renderer design
5. Tests added
6. Test results
7. Existing regression results
8. Dependencies added, if any
9. Any architectural issues discovered
10. Recommended next smallest task

Wait for human review before proceeding further.

4. What Task 4 Actually Builds

4.1 React + TypeScript UI Runtime Foundation

Task 4 introduces the UI Runtime implementation using React and TypeScript. It establishes the runtime-side representation of the v0 UISpec and the renderer that turns an approved specification into React elements.

4.2 Explicit Component Registry

The registry is the executable mapping between an approved UISpec component name and a developer-authored React component.

UISpec component name
↓
Approved Component Registry
↓
Developer-authored React component

The v0 registry contains exactly these nine approved components:

- Hero
- Heading
- Text
- FeatureCard
- FeatureGrid
- CTA
- Alert
- InfoPanel
- ProgressIndicator

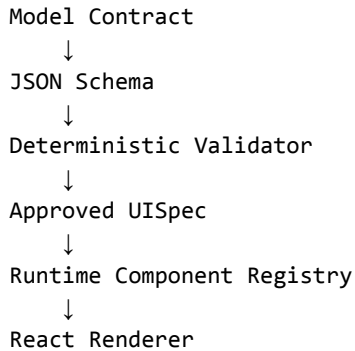
4.3 Renderer

The renderer resolves components only through the explicit registry. If an unsupported component name is encountered, the renderer rejects it rather than attempting to discover or dynamically load a component.

The renderer recursively handles approved child structures and uses developer-authored React components. It does not generate React, HTML, JavaScript, or executable code from model output.

5. Model Contract vs Component Registry

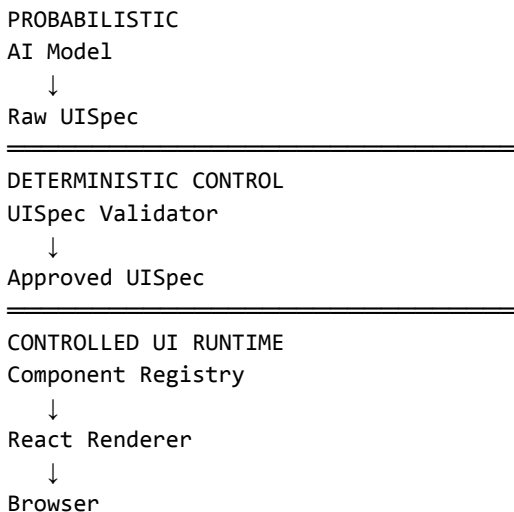
A key enterprise engineering distinction is that the JSON Schema and the component registry are related, but they are not the same artifact.



The JSON Schema defines what the model is allowed to propose. The registry defines which developer-authored React components the runtime is actually willing to execute/render. The registry must not become a second independent model contract.

6. Security and Control Boundaries

Task 4 is primarily about controlling the last step between structured model output and browser UI.



The central principle is: AI can propose; the system decides what is allowed.

6.1 Explicit Registry Instead of Dynamic Loading

The renderer does not use model-supplied component names to dynamically import modules, construct executable code, or perform arbitrary module lookup. The registry is a fixed developer-authored map.

6.2 No eval / Function / Arbitrary HTML Execution

The implementation deliberately avoids mechanisms that would turn model-controlled strings into executable JavaScript or arbitrary HTML. User/model text is rendered as normal React text rather than being treated as executable markup.

6.3 Controlled Props

Components use declared props rather than forwarding arbitrary model-controlled properties to DOM elements. This reduces the risk that an unexpected model property becomes a browser-side capability.

6.4 Unknown Components Are Rejected

If a UISpec contains an unsupported component such as an invented lframe component, the renderer does not attempt to render it. It raises the defined unknown-component error.

7. Deterministic Tests

Task 4 created 19 deterministic tests across four areas:

- Registry integrity tests — 3
- Rendering tests — 5
- Security tests — 8
- Determinism tests — 3

Total: 19 tests, all passing.

7.1 What the Tests Demonstrate

- The registry contains only the approved v0 components.
- Valid UISpec structures render successfully.
- Unsupported component names are rejected.
- The same input produces deterministic rendering behaviour.
- The renderer does not mutate the input UISpec.
- Text is rendered safely rather than executed as HTML.
- Unknown props are not forwarded to the DOM.
- dangerouslySetInnerHTML is not used.
- eval is not used.
- Function constructors are not used.
- Dynamic imports are not used for model-controlled component resolution.

8. Regression Verification

The Task 4 implementation was also checked against the earlier increments:

- Task 4 renderer tests: 19 passed.
- Task 3 Python validator tests: 24 passed.
- Task 2 JSON Schema verification: 12/12 checks passed.
- No runtime model integration was introduced.
- No FastAPI runtime was introduced.
- No authentication, document processing, risk AI, approval workflow, or production infrastructure was introduced.

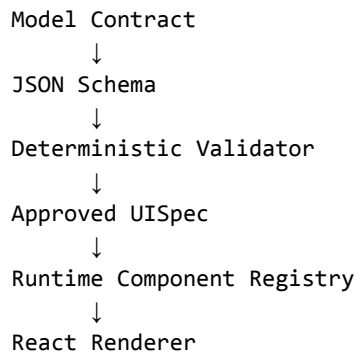
9. Dependencies Introduced

Task 4 introduced the following UI development dependencies:

- React ^18.3.1
- react-dom ^18.3.1
- @types/react ^18.3.3
- @types/react-dom ^18.3.0
- TypeScript ^5.5.3
- Vitest ^2.1.1

The implementation did not introduce React Testing Library, jsdom, a bundler, a linter, FastAPI, an AI model SDK, a database library, or production infrastructure.

10. Task 4 Files



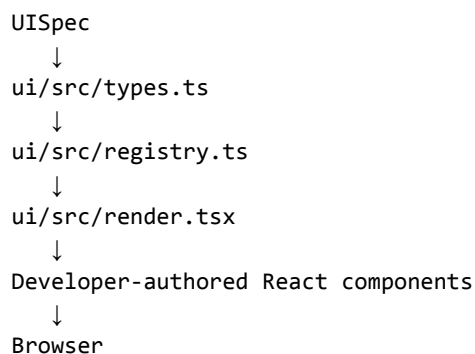
Task 4 creates the React-side controlled rendering capability through `ui/src/registry.ts` and `ui/src/render.tsx`.

Task 3 created the deterministic Python validator in `domain/validator.py`.

Task 2 created the machine-checkable UISpec contract in `specs/schemas/ui-spec.schema.json`.

Relationship to the Earlier Tasks

The registry and renderer are runtime implementation controls. They do not replace the model contract or the deterministic validator created in earlier tasks.



How the Files Work Together

The four test files contain 19 tests in total: 3 registry tests + 5 rendering tests + 8 security tests + 3 determinism tests = 19 tests.

Task 4 Test Summary

- `ui/tests/determinism.test.tsx` — Verifies deterministic renderer behaviour and input non-mutation. 3 tests.

- ui/tests/security.test.tsx — Verifies security boundaries such as unsupported component rejection, safe text rendering, controlled DOM props, and absence of dangerous execution mechanisms. 8 tests.
- ui/tests/render.test.tsx — Verifies normal rendering behaviour for valid UISpec structures. 5 tests.
- ui/tests/registry.test.ts — Verifies registry integrity and the approved component vocabulary. 3 tests.

Test Files

- ui/src/render.tsx — Implements the controlled UISpec renderer. It resolves components through the approved registry, renders approved child structures, and rejects unsupported components.
- ui/src/registry.ts — Contains the explicit developer-controlled component registry. It maps the approved UISpec component names to developer-authored React components.
- ui/src/errors.ts — Defines the renderer error used when an unsupported component is encountered, such as an invented iframe component.
- ui/src/types.ts — Defines the TypeScript runtime representation of the v0 UISpec, including component names, component props, CTA actions, and related types. It does not replace the JSON Schema.

Source / Renderer Files

- ui/README.md — Provides documentation for the UI Runtime.
- ui/vitest.config.ts — Defines the Vitest test configuration.
- ui/tsconfig.json — Defines the TypeScript compiler configuration.
- ui/package.json — Defines the React/TypeScript UI project and its dependencies.

Configuration Files

```

ui/
├── package.json
├── tsconfig.json
├── vitest.config.ts
├── README.md
├── src/
│   ├── types.ts
│   ├── errors.ts
│   ├── registry.ts
│   └── render.tsx
└── tests/
    ├── registry.test.ts
    ├── render.test.tsx
    ├── security.test.tsx
    └── determinism.test.tsx

```

Task 4 File Structure

11. Important Observations from Task 4

11.1 Contract terminology was preserved

An illustrative UISpec in the Task 4 requirements used different example terminology such as 'type' and version '0.1'. The implementation correctly followed the actual existing v0 contract, which uses 'component' and version 'v0'. This reinforces the rule that the existing specification is the source of truth.

11.2 Static security scanning required careful test design

A static security scan initially detected a dangerous keyword inside a test/documentation string. The test was adjusted to ignore comments when performing that scan; this did not change the renderer's runtime behaviour.

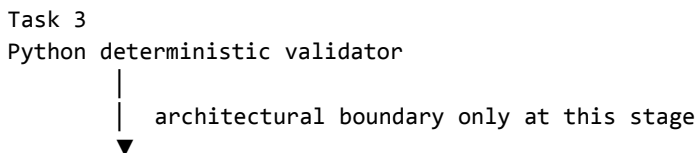
11.3 Package housekeeping

The UI implementation produced the normal package installation artifacts, including `node_modules` and `package-lock.json`. `node_modules` should not be committed. `Package-lock.json` is normally useful for reproducible dependency installation. This is checkpoint housekeeping rather than a new application feature.

12. The Important Clarification Before Checkpoint

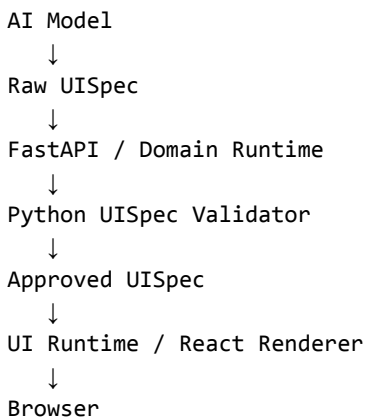
Task 4 should be checkpointed as a controlled renderer increment, but the architecture must not be described as if the Task 3 Python validator already calls the React renderer through a live API.

Current physical implementation:



Task 4
React controlled renderer

The future runtime flow is expected to be:



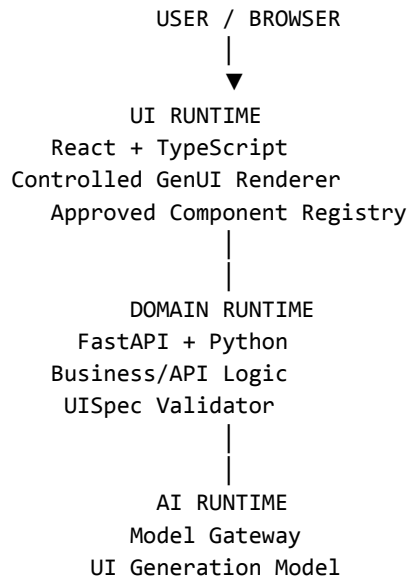
That physical integration belongs to a later task. Keeping it out of Task 4 preserves the incremental architecture and avoids prematurely introducing an API boundary before the model gateway and domain runtime are ready.

13. What Task 4 Does NOT Build

- FastAPI application runtime
- Model gateway
- Runtime AI model integration
- Authentication
- Customer registration

- Document processing
- Risk AI
- Human approval workflow
- Production infrastructure
- Python-validator-to-React API integration
- AI evaluations
- Feedback loop

14. Enterprise Architecture View After Task 4



At this checkpoint, the UI Runtime has a controlled renderer and the Domain Runtime has a deterministic validator, but the physical runtime connection between them has intentionally not yet been implemented.

15. The Key Engineering Lesson

Task 4 demonstrates that Generative UI does not require the model to generate executable UI code.

Model proposes a structured description



Validator decides whether it complies



Registry decides which implementation is available



Developer-authored React components render it

This creates a controlled separation between model-generated intent and executable application capability.

The core principle remains:

AI can propose. The system decides what is allowed.

16. Task 4 Checkpoint Readiness

Based on the implementation report, Task 4 is ready for review/checkpointing as a controlled renderer increment, subject to preserving the clarification in Section 12.

- Task 4 implementation scope is limited to the controlled UI Runtime.
- The component registry is explicit and controlled.
- The renderer accepts an approved UISpec rather than raw AI output.
- The implementation avoids dynamic code execution and model-controlled module loading.
- 19 Task 4 tests pass.
- Task 3 regression tests pass.
- Task 2 schema verification passes.
- No physical validator-to-renderer API integration is claimed yet.

17. What Comes Next

The next implementation increment should be chosen only after the Task 4 checkpoint is reviewed and accepted. The likely direction is to establish the smallest runtime path around the renderer and domain layer, while preserving the existing model contract and validation boundary.

One-Sentence Summary

Task 4 turns the approved UISpec contract into a controlled React rendering capability through an explicit component registry, without allowing the AI model to generate or execute arbitrary UI code.